

AdventQuest — Roadmap de développement complète

Stack technique retenue

Couche	Technologie	Rôle
App mobile	React Native (Expo)	UI, navigation, logique métier
Jeux arcade	Phaser.js (via WebView)	Runner, Casse-briques, etc.
State global	Zustand	Progression, hints, fragments
Backend	Supabase	Auth, base de données, leaderboard
Notifications	Expo Notifications	Rappel quotidien
Animations	React Native Reanimated	Transitions, effets
Assets	Figma + MidJourney/DALL-E	Illustrations, icônes

Vue d'ensemble des phases

Phase 0 – Fondations	(2 semaines)
Phase 1 – Squelette app	(3 semaines)
Phase 2 – Jeux RN	(8 semaines)
Phase 3 – Jeux Phaser	(8 semaines)
Phase 4 – Histoire + contenu	(3 semaines)
Phase 5 – Backend complet	(3 semaines)
Phase 6 – Polish + tests	(3 semaines)
Phase 7 – Publication	(2 semaines)
Total MVP (12 jours)	~ 4–5 mois
Total complet (24j)	~ 9–12 mois

PHASE 0 — Fondations (2 semaines)

Objectif : avoir un projet qui tourne, les outils configurés, et l'architecture en place avant d'écrire une seule ligne de jeu.

Étape 0.1 — Environnement de dev

☐ Installer Node.js (v20+), Git, VS Code

- ☐ Installer Expo CLI: `npm install -g expo-cli`
- ☐ Créer le projet: `npx create-expo-app AdventQuest --template`
- ☐ Configurer iOS Simulator (Mac) ou Android Emulator
- ☐ Tester que `npx expo start` fonctionne sur simulateur ET téléphone physique (Expo Go)

Étape 0.2 — Structure des dossiers

```

AdventQuest/
├── app/                                → navigation (Expo Router)
│   ├── (tabs)/
│   │   ├── index.tsx                 → écran Calendrier
│   │   ├── leaderboard.tsx          → écran Classement
│   │   └── profile.tsx               → écran Profil
│   └── game/[day].tsx                → écran de jeu dynamique
├── components/
│   ├── Calendar/                    → grille des 24 cases
│   ├── DayCell/                     → case individuelle
│   ├── GameWrapper/                 → wrapper commun tous jeux
│   └── UI/                           → boutons, badges, modals
├── games/
│   ├── day01_quiz/
│   │   ├── index.tsx                → composant jeu
│   │   └── config.ts                → difficulté, fragment, score max
│   ├── day02_stack/
│   │   ├── index.tsx                → WebView Phaser
│   │   └── game.html                → jeu Phaser autonome
│   └── ... (24 dossiers)
├── store/
│   ├── gameState.ts                 → progression, fragments, hints
│   └── userStore.ts                  → profil, scores, auth
├── services/
│   ├── supabase.ts                  → client Supabase
│   ├── auth.ts                      → connexion/inscription
│   └── leaderboard.ts                → lecture/écriture scores
├── constants/
│   ├── days.ts                      → config des 24 jours (histoire, fragment, etc.)
│   └── theme.ts                      → couleurs, typographies
└── assets/
    ├── images/
    ├── sounds/
    └── fonts/
  
```

Étape 0.3 — Dépendances à installer

```
bash

# Navigation
npx expo install expo-router

# Animations
npx expo install react-native-reanimated

# WebView (pour les jeux Phaser)
npx expo install react-native-webview

# State management
npm install zustand

# Backend
npm install @supabase/supabase-js

# Notifications
npx expo install expo-notifications

# Gestes (swipe snake, etc.)
npx expo install react-native-gesture-handler
```

Étape 0.4 — Supabase — setup backend

- ☐ Créer un compte sur supabase.com
- ☐ Créer un nouveau projet "AdventQuest"
- ☐ Créer les tables (voir Phase 5 pour le détail)
- ☐ Copier les clés API dans un fichier `.env`
- ☐ Tester la connexion depuis l'app

Étape 0.5 — Git + versioning

- ☐ `git init` + premier commit
- ☐ Créer un repo GitHub/GitLab
- ☐ Configurer `.gitignore` (exclure `.env`, `node_modules`)
- ☐ Mettre en place des branches : `main` (stable), `develop` (dev actif), `feature/nom-feature`

PHASE 1 — Squelette de l'application (3 semaines)

Objectif : navigation complète, calendrier visuel, système de compte, store Zustand —

| sans aucun vrai jeu encore.

Étape 1.1 — Navigation (3 jours)

- ☐ Configurer Expo Router avec les 3 onglets (Calendrier, Classement, Profil)
- ☐ Créer l'écran de jeu dynamique (`game/[day].tsx`)
- ☐ Tester la navigation entre tous les écrans
- ☐ Ajouter les transitions d'écran (slide, fade)

Étape 1.2 — Écran Calendrier (5 jours)

- ☐ Grille 4 colonnes, 24 cases
- ☐ 3 états visuels : fait (vert) / aujourd'hui (violet pulsant) / verrouillé (gris)
- ☐ Case spéciale jour 24 (or + glow)
- ☐ Séparateurs de zones narratives
- ☐ Barre de progression fragments (avec marqueur seuil 12)
- ☐ Bandeau histoire (résumé du chapitre en cours)
- ☐ Bottom sheet qui s'ouvre au clic sur une case
- ☐ Animation neige tombante en arrière-plan
- ☐ Vibration + message si case verrouillée

Étape 1.3 — GameWrapper (4 jours)

| C'est le composant le plus important — tous les jeux passent par lui.

- ☐ Afficher le titre + jour + fragment à gagner
- ☐ Timer (si le jeu en a un)
- ☐ Barre de hints (afficher combien il en reste, bouton utiliser)
- ☐ Écran de victoire : animation confettis + fragment révélé + score
- ☐ Écran d'échec : message narratif + bouton rejouer (si même jour)
- ☐ Communication avec le store (enregistrer score, fragment, tentatives)
- ☐ Support WebView pour les jeux Phaser (passer/recevoir messages)

```
typescript
```

```
// Interface que TOUS les jeux doivent respecter
interface GameResult {
  success: boolean;
  score: number;
  timeSpent: number;
}

// Chaque jeu appelle ça quand il se termine
onGameEnd(result: GameResult);
```

Étape 1.4 — Store Zustand (3 jours)

typescript

```
// gameState.ts – tout l'état de progression
{
  currentDay: number,          // jour actif (depuis date serveur)
  fragments: { [day]: boolean }, // fragment obtenu ou non
  bestScores: { [day]: number }, // meilleur score par jour
  attempts: { [day]: number },  // nb tentatives aujourd'hui
  hints: number,               // hints disponibles
  dayLocked: { [day]: boolean }, // verrouillé après minuit

  // Actions
  canPlay(day): boolean,
  finishAttempt(day, score, success): void,
  useHint(): void,
  advanceDay(newDay): void,
  totalFragments(): number,      // pour vérifier seuil 12
  bossUnlocked(): boolean,      // fragments >= 12
}
```

Étape 1.5 — Écran Classement (3 jours)

- ☐ Podium visuel top 3
- ☐ Liste complète avec rang, avatar, score, streak
- ☐ Toggle Amis / Global
- ☐ Données mockées (pas encore connectées à Supabase)

Étape 1.6 — Écran Profil (2 jours)

- ☐ Avatar + pseudo
- ☐ Statistiques (score total, streak, fragments)
- ☐ Grille des fragments collectés
- ☐ Section règles du jeu
- ☐ Bouton déconnexion

Étape 1.7 — Auth basique (3 jours)

- ☐ Écran d'inscription (email + pseudo + mot de passe)
 - ☐ Écran de connexion
 - ☐ Persistance de session (rester connecté)
 - ☐ Redirection auto si déjà connecté
-

PHASE 2 — Jeux React Native (8 semaines)

┃ Objectif : implémenter les 9 jeux RN un par un, chacun branché sur le GameWrapper.

Ordre recommandé : du plus simple au plus complexe.

Jeu 1 — Quiz Noël (jour 1) — 3 jours

Mécanique : 5 questions QCM, 4 choix, feedback immédiat, score = bonnes réponses × temps restant.

- ☐ Composant question avec 4 boutons
- ☐ Animation feedback (vert/rouge sur la bonne/mauvaise réponse)
- ☐ Score final + transmission au GameWrapper
- ☐ Contenu : écrire les 30+ questions (pour tous les jours quiz)
- ☐ Système de sélection aléatoire sans répétition

Jeu 2 — Dessin connecté (jour 6) — 3 jours

Mécanique : points numérotés en SVG, joueur relie dans l'ordre, révèle une image.

- ☐ SVG avec points positionnés manuellement
- ☐ Détection du toucher sur le bon point suivant
- ☐ Ligne qui se trace progressivement
- ☐ Révélation de l'image finale (fade in)
- ☐ Créer 3 dessins différents (flocon, étoile, renne)

Jeu 3 — Pipe Puzzle (jour 10) — 4 jours

Mécanique : grille de segments, le joueur tourne chaque pièce, doit connecter entrée → sortie.

- ☐ Grille de tuiles rotatives (4 types : droit, coude, T, croix)
- ☐ Rotation au tap (90° à chaque tap)
- ☐ Algorithme de validation (chemin connecté ?)
- ☐ Animation flux magique quand résolu
- ☐ Générer 5 puzzles de difficulté croissante

Jeu 4 — Sudoku (jours 5, 12, 17) — 6 jours

Mécanique : grille classique, le joueur tape un chiffre dans chaque case.

- ☐ Grille 4×4 puis 6×6 (même composant, paramétrable)
- ☐ Clavier numérique personnalisé (1-4 ou 1-6)
- ☐ Validation ligne / colonne / bloc en temps réel
- ☐ Mise en surbrillance des conflits
- ☐ Cases pré-remplies non modifiables (couleur différente)

- ☐ Hint : révèle une case
- ☐ Générer ou hard-coder 10 puzzles valides par taille

Jeu 5 — Memory Séquence / Simon (jour 18) — 4 jours

Mécanique : 4 symboles clignotent dans un ordre, le joueur reproduit la séquence.

- ☐ 4 boutons/zones colorées avec animation "flash"
- ☐ Séquence qui s'allonge à chaque tour réussi
- ☐ Délai entre les flashes qui diminue
- ☐ Game over si mauvais symbole
- ☐ Score = longueur de séquence atteinte
- ☐ Hint : rejoue la séquence

Jeu 6 — Snake Croque-Pommes (jour 13) — 4 jours

Mécanique : grille 10×10, serpent qui avance, pas de mort, max pommes en 45 secondes.

- ☐ Grille rendue en View/grid
- ☐ Tick de jeu avec setInterval (vitesse fixe)
- ☐ Swipe pour changer de direction (react-native-gesture-handler)
- ☐ Boutons directionnels alternatifs (pour les non-swipeurs)
- ☐ Serpent traverse les murs sans mourir
- ☐ Pomme réapparaît ailleurs après être mangée
- ☐ Timer 45 secondes, score = pommes mangées

Jeu 7 — Labyrinthe (jour 14) — 4 jours

Mécanique : grille 10×10, le joueur déplace un personnage case par case.

- ☐ Algorithme de génération de labyrinthe (recursive backtracking)
- ☐ Rendu grille avec murs
- ☐ Déplacement par swipe OU boutons directionnels
- ☐ Murs mobiles à intervalle régulier (version difficile)
- ☐ Détection arrivée à la sortie

Jeu 8 — Nonogram / Picross (jours 15, 23) — 5 jours

Mécanique : grille à colorier selon des indices numériques sur lignes et colonnes.

- ☐ Rendu grille avec indices sur les axes
- ☐ Tap pour colorier / décolorier une case
- ☐ Validation automatique quand grille complète
- ☐ Hint : révèle une ligne entière
- ☐ Créer 5 images encodées en Nonogram (flocon, sapin, étoile, bonhomme de neige, cœur)
- ☐ Grille 5×5 puis 7×7

Jeu 9 — Solitaire Klondike (jour 11) — 8 jours

Mécanique : jeu de cartes classique, tirer des cartes, construire les 4 piles par couleur de As à Roi.

! Le plus complexe — à commencer tôt, prévoir du temps.

- ☐ Modèle de données du jeu (deck, colonnes, piles de fondation, pioche)
- ☐ Algorithme de distribution initiale
- ☐ Drag & drop des cartes entre colonnes
- ☐ Détection des mouvements valides
- ☐ Flip automatique de la carte du dessus
- ☐ Pioche (tirage 1 carte, recyclage du talon)
- ☐ Détection victoire (4 piles complètes)
- ☐ Hint : surligne un mouvement possible

Jeu 10 — Jeu de Rythme (jour 21) — 5 jours

Mécanique : notes tombent du haut, le joueur tape en bas au bon moment.

- ☐ Rendu des notes qui tombent (Animated ou Reanimated)
- ☐ Zones de tap en bas (4 colonnes)
- ☐ Synchronisation avec un fichier audio
- ☐ Feedback visuel + score (Perfect / Good / Miss)
- ☐ Créer 2 partitions de mélodies de Noël
- ☐ Hint : ralentit le tempo 10 secondes

PHASE 3 — Jeux Phaser (8 semaines)

Objectif : implémenter les 10 jeux Phaser, chacun dans une page HTML autonome, intégré via WebView.

Setup commun Phaser (2 jours)

- ☐ Créer un template (`phaser-game-template.html`) réutilisable
- ☐ Communication RN ↔ Phaser via (`postMessage`)
- ☐ Template GameWrapper côté Phaser (timer, score affiché, bouton hint)
- ☐ Adapter le design (couleurs dark, thème Noël)

```
javascript
```



```
// Dans chaque jeu Phaser – envoyer le résultat à React Native
function endGame(score, success) {
  window.ReactNativeWebView.postMessage(JSON.stringify({
    type: 'GAME_OVER',
    score: score,
    success: success
  }));
}

// Recevoir un hint de React Native
window.addEventListener('message', (event) => {
  const data = JSON.parse(event.data);
  if (data.type === 'USE_HINT') activateHint();
});
```

Jeu Phaser 1 — Stack (jour 2) — 3 jours

- ☐ Plateforme qui oscille gauche-droite à vitesse croissante
- ☐ Tap = drop la plateforme
- ☐ Découpage de la portion qui dépasse
- ☐ Tour qui monte (caméra qui suit)
- ☐ Game over si plateforme trop petite
- ☐ Score = nombre d'étages empilés

Jeu Phaser 2 — Piano Tiles Noël (jour 3) — 4 jours

- ☐ 4 colonnes, tuiles noires qui descendent
- ☐ Tap sur la tuile noire = point, tap sur blanc = game over
- ☐ Vitesse croissante au fil du temps
- ☐ Son de note à chaque tap
- ☐ Mélodie de Noël synchronisée avec les tuiles
- ☐ Score = tuiles tapées × combo

Jeu Phaser 3 — Space Invaders Noël (jours 4, 22) — 5 jours

- ☐ Grille d'ennemis (créatures de Grimnoir) qui descend
- ☐ Vaisseau (traîneau) qui se déplace gauche/droite
- ☐ Tir de boules de neige (tap/bouton)
- ☐ Vagues de plus en plus rapides
- ☐ Ennemis ripostent (projectiles à éviter)
- ☐ Score = ennemis détruits × bonus vague

Jeu Phaser 4 — Fruit Ninja Noël (jour 7) — 4 jours

- ☐ Objets qui montent/tombent (ornements, étoiles, cannes)
- ☐ Swipe détecté → tranche les objets touchés
- ☐ Explosion/particules à la découpe
- ☐ Bombes à éviter (noir/rouge)
- ☐ Score = objets tranchés, pénalité si bombe

Jeu Phaser 5 — Bubble Shooter (jours 8, 22) — 5 jours

- ☐ Grille hexagonale de bulles colorées en haut
- ☐ Canon en bas, joueur vise et tire
- ☐ Matching 3+ bulles de même couleur = éclatement
- ☐ Bulles qui tombent par gravité après éclatement
- ☐ Game over si grille descend trop bas
- ☐ Score = bulles éclatées × combo

Jeu Phaser 6 — Runner de Noël (jour 9) — 5 jours

- ☐ Personnage qui court automatiquement
- ☐ Bouton saut + bouton glisse
- ☐ Génération procédurale des obstacles (racines, stalactites)
- ☐ Collecte de fragments en vol (bonus score)
- ☐ Timer 60 secondes
- ☐ Vitesse qui augmente progressivement
- ☐ Score = distance + fragments collectés

Jeu Phaser 7 — Whack-a-Mole (jour 16) — 3 jours

- ☐ Grille de trous (3×3 ou 4×4)
- ☐ Lutins qui surgissent aléatoirement
- ☐ Tap = coup, animation marteau
- ☐ Faux fragments (ne pas taper) → pénalité
- ☐ Timer 45 secondes
- ☐ Score = lutins tapés – pénalités

Jeu Phaser 8 — Dodge Ball (jours 19, 23) — 4 jours

- ☐ Personnage en bas de l'écran, déplacement gauche/droite
- ☐ Projectiles de Grimnoir tombent du haut
- ☐ Swipe ou joystick virtuel pour se déplacer
- ☐ Vitesse et fréquence croissantes
- ☐ Timer 60-90 secondes
- ☐ Score = temps survécu × multiplicateur

Jeu Phaser 9 — Casse-briques (jours 20, 24) — 5 jours

- ☐ Grille de blocs de glace (couleurs variées)
- ☐ Raquette en bas, contrôle par toucher
- ☐ Balle qui rebondit (physique Arcade Phaser)
- ☐ Blocs bonus (balle double, raquette large, +10 secondes)
- ☐ Blocs malus (inversion commandes, balle rapide)
- ☐ Game over si balle tombe en bas

Jeu Phaser 10 — Boss Final phase 1 & 2 (jour 24) — 3 jours

- ☐ Adapter Stack (vitesse très élevée, 20 étages)
 - ☐ Adapter Casse-briques (1 vie, malus fréquents)
 - ☐ Séquençage des 3 phases dans le GameWrapper
 - ☐ Animation de transition entre phases
 - ☐ Animation finale de reconstruction du Cœur de Noël
-

PHASE 4 — Histoire & contenu (3 semaines)

┃ Objectif : tous les textes, images, sons, et données de contenu.

Étape 4.1 — Textes narratifs (4 jours)

- ☐ Rédiger les 24 textes d'intro (bottom sheet de chaque jour)
- ☐ Rédiger les 6 textes de zone (bandeau narratif)
- ☐ Rédiger les textes de victoire et d'échec par zone (ton différent)
- ☐ Écrire 50+ questions quiz Noël (avec vérification des réponses)
- ☐ Écrire les dialogues du Renard d'Hiver (jours 10, 11)

Étape 4.2 — Illustrations (8 jours)

┃ Via MidJourney, DALL-E 3, ou un illustrateur freelance.

- ☐ Illustration de fond par zone (6 illustrations)
- ☐ Avatar Gardien des Fêtes
- ☐ Avatar Grimnoir
- ☐ Avatar Renard d'Hiver
- ☐ Icônes des 24 fragments
- ☐ Sprites pour les jeux Phaser (lutins, ornements, créatures)
- ☐ Illustration "Cœur de Noël reconstruit" (écran final)

Étape 4.3 — Sons et musique (4 jours)

▮ [Freesound.org](https://freesound.org) pour les effets, [Pixabay](https://pixabay.com) pour la musique libre de droits.

- ☐ Musique de fond (ambiante, Noël, douce)
- ☐ Son d'ouverture de case (pop magique)
- ☐ Son de victoire (fanfare courte)
- ☐ Son d'échec (accord mineur)
- ☐ Sons des jeux (boules de neige, bris de glace, notes piano)
- ☐ Son de hint utilisé
- ☐ Son de fragment obtenu (cristallin)

Étape 4.4 — Données de contenu centralisées

typescript

```
// constants/days.ts – toutes les données des 24 jours au même endroit
export const DAYS_CONFIG = {
  1: {
    game: 'quiz',
    difficulty: 'easy',
    fragment: { name: 'Éclat de lumière', icon: '✦' },
    story: {
      intro: "Une vieille statue gardienne te pose 5 questions...",
      success: "La lumière revient dans les yeux de la statue !",
      failure: "La statue secoue la tête, déçue..."
    },
    hints: { count: 1, type: 'eliminate_answers' }
  },
  // ... 23 autres jours
}
```

PHASE 5 — Backend Supabase complet (3 semaines)

Étape 5.1 — Schéma de base de données (3 jours)

sql

```
-- Utilisateurs
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
  username TEXT UNIQUE NOT NULL,
  avatar_url TEXT,
  created_at TIMESTAMP DEFAULT NOW()
);

-- Progression par joueur
CREATE TABLE player_progress (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  day INTEGER NOT NULL,          -- 1 à 24
  completed BOOLEAN DEFAULT FALSE,
  fragment_won BOOLEAN DEFAULT FALSE,
  best_score INTEGER DEFAULT 0,
  attempts INTEGER DEFAULT 0,
  completed_at TIMESTAMP,
  UNIQUE(user_id, day)
);

-- Scores pour le leaderboard
CREATE TABLE leaderboard (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID REFERENCES users(id),
  total_score INTEGER DEFAULT 0,
  fragments_count INTEGER DEFAULT 0,
  streak INTEGER DEFAULT 0,
  updated_at TIMESTAMP DEFAULT NOW()
);

-- Hints
CREATE TABLE player_hints (
  user_id UUID PRIMARY KEY REFERENCES users(id),
  hints_available INTEGER DEFAULT 0,
  hints_used INTEGER DEFAULT 0
);

-- Amis
CREATE TABLE friendships (
  user_id UUID REFERENCES users(id),
  friend_id UUID REFERENCES users(id),
  status TEXT DEFAULT 'pending', -- pending / accepted
```

```
PRIMARY KEY (user_id, friend_id)
);
```

Étape 5.2 — Sécurité anti-triche (3 jours)

- ☐ Row Level Security (RLS) sur toutes les tables
- ☐ Edge Function pour valider les scores côté serveur
 - Vérifier que `day === today` (date serveur, pas client)
 - Vérifier que le score est dans une plage plausible
 - Vérifier que le joueur n'a pas déjà soumis pour ce jour
- ☐ Rate limiting sur les soumissions de score

typescript

```
// services/supabase.ts – soumission sécurisée
async function submitScore(day: number, score: number, success: boolean) {
  const { data, error } = await supabase.functions.invoke('validate-score', {
    body: { day, score, success, timestamp: Date.now() }
  });
  // Le serveur valide tout – le client ne peut pas tricher
}
```

Étape 5.3 — Leaderboard temps réel (3 jours)

typescript

```
// Subscription temps réel Supabase
supabase
  .channel('leaderboard')
  .on('postgres_changes', {
    event: 'UPDATE',
    schema: 'public',
    table: 'leaderboard'
  }, (payload) => {
    updateLocalLeaderboard(payload.new);
  })
  .subscribe();
```

Étape 5.4 — Notifications push (2 jours)

- ☐ Configuration Expo Notifications
- ☐ Permission au premier lancement
- ☐ Notification quotidienne à l'heure choisie ("🎄 Nouveau fragment disponible!")
- ☐ Notification si le streak est en danger ("⚠️ Plus que 2h pour sauvegarder ton streak!")

- ☐ Désactivable dans les réglages

Étape 5.5 — Système d'amis (3 jours)

- ☐ Recherche par pseudo
 - ☐ Envoi/acceptation de demande d'ami
 - ☐ Leaderboard "Amis" filtré
 - ☐ Notification quand un ami te dépasse au classement
-

PHASE 6 — Polish & tests (3 semaines)

Étape 6.1 — Équilibrage des jeux (5 jours)

- ☐ Playtesting de chaque jeu (faire jouer 5+ personnes)
- ☐ Ajuster les timers (ni trop facile ni frustrant)
- ☐ Vérifier la difficulté progressive entre zones
- ☐ Vérifier que chaque jeu est "faisable sans hint" même en fin de calendrier
- ☐ Calibrer les scores pour que le leaderboard soit équitable

Étape 6.2 — Animations & micro-interactions (4 jours)

- ☐ Animation d'ouverture de case (rotation + particules)
- ☐ Animation de fragment obtenu (cristal qui se place dans le Cœur)
- ☐ Confettis lors des victoires
- ☐ Glow pulsant sur la case du jour
- ☐ Transition entre les écrans
- ☐ Animation de chargement thématique (flocon qui tourne)

Étape 6.3 — Tests (5 jours)

- ☐ Tester sur iOS (iPhone SE, iPhone 15, iPad)
- ☐ Tester sur Android (petit écran, grand écran, tablette)
- ☐ Tester la connexion lente / hors ligne
- ☐ Tester le changement de jour à minuit (transition currentDay)
- ☐ Tester l'inscription, connexion, déconnexion
- ☐ Tester les cas limites (0 hint, 0 fragment, boss verrouillé)
- ☐ Tester la date serveur vs date téléphone (anti-triche)

Étape 6.4 — Accessibilité & performance (3 jours)

- ☐ Textes lisibles (contraste suffisant)
- ☐ Tailles de police adaptées (Dynamic Type iOS)
- ☐ Pas de lag sur les jeux Phaser (tester sur vieux téléphones)

- ☐ Temps de chargement < 3 secondes au lancement
 - ☐ Images compressées (WebP)
-

PHASE 7 — Publication (2 semaines)

Étape 7.1 — Préparation assets stores (3 jours)

- ☐ Icône app (1024×1024 PNG, pas de transparence)
- ☐ Splash screen (thème Noël)
- ☐ Screenshots iOS (6.7", 6.1", iPad)
- ☐ Screenshots Android (téléphone + tablette)
- ☐ Description app (FR + EN)
- ☐ Texte "Quoi de neuf" pour les mises à jour

Étape 7.2 — Build de production (2 jours)

```
bash

# Build iOS
eas build --platform ios --profile production

# Build Android
eas build --platform android --profile production
```

- ☐ Configurer EAS Build (Expo Application Services)
- ☐ Certificats iOS (Apple Developer Program — 99\$/an)
- ☐ Keystore Android (généré une fois, conservé précieusement)

Étape 7.3 — Soumission App Store (3 jours)

- ☐ Créer la fiche sur App Store Connect
- ☐ Remplir toutes les métadonnées
- ☐ Déclarer les usages de données (privacy labels)
- ☐ Soumettre pour review (délai moyen : 24-48h, peut aller jusqu'à 7 jours)

Étape 7.4 — Soumission Google Play (2 jours)

- ☐ Créer la fiche sur Google Play Console
 - ☐ Remplir le questionnaire de contenu
 - ☐ Soumettre en "test interne" d'abord, puis production
 - ☐ Délai review : 24h en général
-

Recommandation MVP pour Noël 2025

Si le temps manque, voici les 12 jours prioritaires à implémenter en premier :

Jour	Jeu	Priorité
1	Quiz Noël	★ Indispensable (introduction)
2	Stack	★ Signature du jeu
4	Space Invaders	★ Action dès le début
5	Sudoku 4×4	★ Classique attendu
7	Fruit Ninja	★ Fun immédiat
9	Runner	★ Showpiece Phaser
11	Solitaire	★ Jeu de carte
13	Snake	★ Nostalgie
16	Whack-a-Mole	★ Zone 4
19	Dodge Ball	★ Tension
20	Casse-briques	★ Final zone 4
24	Boss (2 phases seulement)	★ Conclusion

Les 12 autres jours affichent "À venir — Saison 2" avec un design mystérieux. Ça crée même de la hype pour l'année suivante.